

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

кафедра програмних засобів

ЗВІТ

з лабораторної роботи № 2

з дисципліни «Агентно-орієнтоване програмування» на тему:

**«СТВОРЕННЯ МУЛЬТИАГЕНТНИХ СИСТЕМ ДЛЯ
РОЗПОДІЛЕНИХ ОБЧИСЛЕНЬ У SPADE»**

Виконав:

ст. гр. КНТ-113сп

Іван ЩЕДРОВСЬКИЙ

Прийняв:

Доцент

Олена ПОДКОВАЛІХІНА

1 Мета роботи

Ознайомитися з основами мультиагентних систем (MAC) для розподілених обчислень. Навчитися використовувати фреймворк SPADE для організації обчислювальних агентів. Реалізувати систему агентів, що взаємодіють для розв'язання завдань у паралельному середовищі

2 Завдання до лабораторної роботи

Розробити мультиагентну систему для знаходження найкоротшого шляху в графі

3 Виконання лабораторної роботи

Для виконання цієї лабораторної роботи було створено три аккаунти на сервері XMPP. Також було переглянути методичні вказівки та запущено код згідно з прикладами.

Для реалізації застосунку було створено трьох агентів.

Перший агент виступає в ролі керівника, який роздає завдання. Цей агент створює графи та після відправляє їх до інших агентів, які будуть розраховувати найкоротший шлях. Передача інформації буде відбуватись в форматі JSON. Коли інші агенти знайдуть відповідь вони надсилають її до першого, і він вже її показує.

Другий та третій агенти будуть приймати запити від першого, перетворювати дані передані через мережу на внутрішні представлення, знаходити найкоротший шлях та повертати його до агента-керівника.

На рисунку 1 показано приклад запуску агента, який виступає в ролі менеджера.

```

Manager: Hi!
Manager: Send graph 1 to agent ltlaitoff-2@xmpp.jp. Expected result: 3
Manager: Send graph 2 to agent ltlaitoff-3@xmpp.jp. Expected result: 13
Manager: Send graph 3 to agent ltlaitoff-2@xmpp.jp. Expected result: 3
Manager: Send graph 4 to agent ltlaitoff-3@xmpp.jp. Expected result: 11
Manager: Send graph 5 to agent ltlaitoff-2@xmpp.jp. Expected result: 8
Manager: Waiting for messages!
Manager: Message from "ltlaitoff-2@xmpp.jp/36929448802041386375938969026" received! Result = {"id": 1, "result": 3}
Manager: Waiting for messages!
Manager: Message from "ltlaitoff-3@xmpp.jp/71442791800014495085938971202" received! Result = {"id": 2, "result": 13}
Manager: Waiting for messages!
Manager: Message from "ltlaitoff-2@xmpp.jp/36929448802041386375938969026" received! Result = {"id": 3, "result": 3}
Manager: Waiting for messages!
Manager: Message from "ltlaitoff-3@xmpp.jp/71442791800014495085938971202" received! Result = {"id": 4, "result": 11}
Manager: Waiting for messages!
Manager: Message from "ltlaitoff-2@xmpp.jp/36929448802041386375938969026" received! Result = {"id": 5, "result": 8}
Manager: Waiting for messages!
Manager: Bye!

```

Рисунок 1 – Запуск агента-менеджера

На рисунку 2 та рисунку 3 показані приклади запуску агентів, які виконують обчислення

```

Graph 1: Hi!
Graph 1: Waiting for messages!
Graph 1: Message from "ltlaitoff@xmpp.jp/103825086214445818285938971458" received! Value = {"id": 1, "from_node": "A", "to_node": "D", "graph": {"A": {"B": 1, "D": 4}, "B": {"C": 3, "D": 2}, "C": {"D": 0}, "D": {}}, "expected_length": 3}
Graph 1: Result is = 3
Graph 1: Send message '{"id": 1, "result": 3}'
Graph 1: Waiting for messages!
Graph 1: Message from "ltlaitoff@xmpp.jp/103825086214445818285938971458" received! Value = {"id": 3, "from_node": "A", "to_node": "C", "graph": {"A": {"B": 1, "C": 4}, "B": {"A": 1, "C": 3, "D": 1}, "C": {"A": 4, "B": 2, "D": 1}, "D": {"B": 5, "C": 1}}, "expected_length": 3}
Graph 1: Result is = 3
Graph 1: Send message '{"id": 3, "result": 3}'
Graph 1: Waiting for messages!
Graph 1: Message from "ltlaitoff@xmpp.jp/103825086214445818285938971458" received! Value = {"id": 5, "from_node": "A", "to_node": "F", "graph": {"A": {"B": 4, "C": 3, "D": 7}, "B": {"E": 2, "F": 6}, "C": {"B": 1, "E": 5, "G": 4}, "D": {"C": 2, "G": 3}, "E": {"H": 3, "I": 7}, "F": {"B": 1, "I": 2, "J": 6}, "G": {"F": 3, "J": 4}, "H": {"I": 2}, "I": {"C": 1, "J": 5}, "J": {"A": 9}}, "expected_length": 8}
Graph 1: Result is = 8
Graph 1: Send message '{"id": 5, "result": 8}'
Graph 1: Waiting for messages!
Graph 1: Bye!

```

Рисунок 2 – Запуск першого агента, який виконує обчислення

```

Graph 2: Hi!
Graph 2: Waiting for messages!
Graph 2: Message from "ltlaitoff@xmpp.jp/103825086214445818285938971458" received! Value = {"id": 2, "from_node": "D", "to_node": "T", "graph": {"D": {"O": 10}, "O": {"T": 3, "P": 2}, "T": {"C": 1}, "P": {}, "C": {}}, "expected_length": 13}
Graph 2: Result is = 13
Graph 2: Send message "{"id": 2, "result": 13}"
Graph 2: Waiting for messages!
Graph 2: Message from "ltlaitoff@xmpp.jp/103825086214445818285938971458" received! Value = {"id": 4, "from_node": "A", "to_node": "J", "graph": {"A": {"B": 2, "C": 6, "E": 3}, "B": {"D": 4, "F": 7}, "C": {"B": 1, "E": 5, "G": 2}, "D": {"H": 3}, "E": {"D": 6, "I": 4}, "F": {"H": 2, "J": 5}, "G": {"F": 1, "J": 3}, "H": {"I": 2}, "I": {"J": 6}, "J": {"A": 8}}, "expected_length": 11}
Graph 2: Result is = 11
Graph 2: Send message "{"id": 4, "result": 11}"
Graph 2: Waiting for messages!
Graph 2: Bye!

```

Рисунок 3 – Запуск другого агента, який виконує обчислення

4 Тест розробленої програми

```

## app.py
import glob

res = glob.glob("*.txt", recursive=False)

with open('codeAll.txt', 'w+', encoding='utf8') as f:
    for i in res:
        with open(i, "r", encoding="utf8") as readFile:
            fileData = readFile.read()
            f.write("## " + i + '\n')
            f.write(fileData)
            f.write('\n\n')

## graph_agent.py
import heapq
import json

from spade.agent import Agent
from spade.behaviour import CyclicBehaviour
from spade.message import Message
from spade.template import Template

# Directed graph representation
class Graph:
    # Data saved in format Dict<from_node_id, Dict<to_node_id, weight>>
    graph = {}

    def __init__(self, graph=None):
        if graph is not None:
            self.graph = graph

    def add_edge(self, from_node, to_node: str, weight: float) -> None:
        if from_node not in self.graph:
            self.graph[from_node] = {}
        if to_node not in self.graph:
            self.graph[to_node] = {}

        self.graph[from_node][to_node] = weight

```

```

def find_path(self, from_node: str, to_node: str):
    distances = {}
    for node in self.graph:
        distances[node] = float("inf")

    heap_queue = [(0, from_node)]

    while heap_queue:
        distance, node = heapq.heappop(heap_queue)

        if distance > distances[node]:
            continue

        for neighbor, weight in self.graph[node].items():
            neighbor_distance = distance + weight

            if neighbor_distance >= distances[neighbor]:
                continue

            distances[neighbor] = neighbor_distance
            heapq.heappush(heap_queue, (neighbor_distance, neighbor))

    return distances[to_node]

result_template = Template()
result_template.set_metadata("performative", "inform")
result_template.set_metadata("language", "FIFA-SL")
result_template.set_metadata("ontology", "RESULT-GRAPH")

graph_json_template = Template()
graph_json_template.set_metadata("performative", "request")
graph_json_template.set_metadata("language", "FIFA-SL")
graph_json_template.set_metadata("ontology", "GRAPH-JSON")

class GraphAgent(Agent):
    # Custom agent identifier. For debug purposes only
    name = "Unknown sender"
    message_time = 10

    # ID of receiver
    manager_id = ""

    def __init__(self, jid: str, password: str, name: str, to_id: str):
        super().__init__(jid, password)
        self.name = name
        self.manager_id = to_id

class MessagesBehaviour(CyclicBehaviour):
    async def run(self):
        print(f"{self.agent.name}: Waiting for messages!")
        message = await self.receive(timeout=self.agent.message_time)

        if not message:
            print(f"{self.agent.name}: Bye!")
            await self.agent.stop()
            return

        assert graph_json_template.match(message), "Message with graph should match graph-json template"

```

```

print(f'{self.agent.name}: Message from \"{message.sender}\" received! Value = {message.body}')

parsed_message = json.loads(message.body)
graph = Graph(parsed_message["graph"])
result = graph.find_path(parsed_message["from_node"], parsed_message["to_node"])

print(f'{self.agent.name}: Result is = {result}')

response_value = json.dumps({"id": parsed_message['id'], "result": result})

message = Message(to=self.agent.manager_id)
message.body = response_value
message.set_metadata("performative", "inform")
message.set_metadata("language", "FIFA-SL")
message.set_metadata("ontology", "RESULT-GRAPH")
assert result_template.match(message), "Result message should match result template"

await self.send(message)
print(f'{self.agent.name}: Send message \"{message.body}\"')

async def setup(self):
    print(f'{self.name}: Hi!')
    self.add_behaviour(self.MessagesBehaviour())

## keys.py
import os
from dotenv import load_dotenv
load_dotenv()

XMPP_ID_ONE=os.getenv('XMPP_ID_ONE')
XMPP_PASSWORD_ONE=os.getenv('XMPP_PASSWORD_ONE')
XMPP_ID_TWO=os.getenv('XMPP_ID_TWO')
XMPP_PASSWORD_TWO=os.getenv('XMPP_PASSWORD_TWO')
XMPP_ID_THREE=os.getenv('XMPP_ID_THREE')
XMPP_PASSWORD_THREE=os.getenv('XMPP_PASSWORD_THREE')

assert XMPP_ID_ONE is not None and XMPP_ID_ONE != "", "XMPP id ONE environment variable is not defined!"
assert XMPP_PASSWORD_ONE is not None and XMPP_PASSWORD_ONE != "", "XMPP password ONE environment variable is not defined!"
assert XMPP_ID_TWO is not None and XMPP_ID_TWO != "", "XMPP id TWO environment variable is not defined!"
assert XMPP_PASSWORD_TWO is not None and XMPP_PASSWORD_TWO != "", "XMPP password TWO environment variable is not defined!"
assert XMPP_ID_THREE is not None and XMPP_ID_THREE != "", "XMPP id THREE environment variable is not defined!"
assert XMPP_PASSWORD_THREE is not None and XMPP_PASSWORD_THREE != "", "XMPP password THREE environment variable is not defined!"

## manager_agent.py
import json
from asyncio import sleep
from typing import List

from spade.template import Template
from spade.agent import Agent
from spade.behaviour import CyclicBehaviour, OneShotBehaviour
from spade.message import Message

```

```

graph_json_template = Template()
graph_json_template.set_metadata("performative", "request")
graph_json_template.set_metadata("language", "FIFA-SL")
graph_json_template.set_metadata("ontology", "GRAPH-JSON")

```

```

result_template = Template()
result_template.set_metadata("performative", "inform")
result_template.set_metadata("language", "FIFA-SL")
result_template.set_metadata("ontology", "RESULT-GRAPH")

```

```

class ManagerAgent(Agent):
    # Custom agent identifier. For debug purposes only
    name = "Unknown receiver"
    message_time = 10

    graphs = []

    def __init__(self, jid: str, password: str, name: str, graph_agent_ids: List[str]):
        super().__init__(jid, password)
        self.name = name
        self.create_graphs()
        self.graph_agent_ids = graph_agent_ids

    def create_graphs(self):
        self.graphs = [
            {
                "id": 1,
                "from_node": "A",
                "to_node": "D",
                "graph": {
                    "A": {"B": 1, "D": 4},
                    "B": {"C": 3, "D": 2},
                    "C": {"D": 0},
                    "D": {},
                },
                "expected_length": 3
            },
            {
                "id": 2,
                "from_node": "D",
                "to_node": "T",
                "graph": {
                    "D": {"O": 10},
                    "O": {"T": 3, "P": 2},
                    "T": {"C": 1},
                    "P": {},
                    "C": {},
                },
                "expected_length": 13
            },
            {
                "id": 3,
                "from_node": "A",
                "to_node": "C",
                "graph": {
                    "A": {"B": 1, "C": 4},
                    "B": {"A": 1, "C": 3, "D": 1},
                    "C": {"A": 4, "B": 2, "D": 1},
                    "D": {"B": 5, "C": 1}
                }
            }
        ]

```

```

    },
    "expected_length": 3
  },
  {
    "id": 4,
    "from_node": "A",
    "to_node": "J",
    "graph": {
      "A": {"B": 2, "C": 6, "E": 3},
      "B": {"D": 4, "F": 7},
      "C": {"B": 1, "F": 5, "G": 2},
      "D": {"H": 3},
      "E": {"D": 6, "I": 4},
      "F": {"H": 2, "J": 5},
      "G": {"F": 1, "J": 3},
      "H": {"I": 2},
      "I": {"J": 6},
      "J": {"A": 8}
    },
    "expected_length": 11
  },
  {
    "id": 5,
    "from_node": "A",
    "to_node": "F",
    "graph": {
      "A": {"B": 4, "C": 3, "D": 7},
      "B": {"E": 2, "F": 6},
      "C": {"B": 1, "F": 5, "G": 4},
      "D": {"C": 2, "G": 3},
      "E": {"H": 3, "I": 7},
      "F": {"E": 1, "I": 2, "J": 6},
      "G": {"F": 3, "J": 4},
      "H": {"I": 2},
      "I": {"G": 1, "J": 5},
      "J": {"A": 9}
    },
    "expected_length": 8
  }
]

```

```

class SendTasksBehaviour(OneShotBehaviour):
    async def run(self):
        for index, value in enumerate(self.agent.graphs):
            to_id_index = index % len(self.agent.graph_agent_ids)
            to_id = self.agent.graph_agent_ids[to_id_index]

            message = Message(to=to_id)
            message.body = json.dumps(value)
            message.set_metadata("performative", "request")
            message.set_metadata("language", "FIFA-SL")
            message.set_metadata("ontology", "GRAPH-JSON")
            assert graph_json_template.match(message), "Message with graph should match graph-json template"

            await self.send(message)
            print(f'{self.agent.name}: Send graph {value["id"]} to agent {to_id}. Expected result: {value["expected_length"]}')

class ReceiveResultBehaviour(CyclicBehaviour):

```


